

Тренинг: AI-driven разработка ИИ- агентов



Эксперты

🏆 Обучат методике AI-driven разработки 🏆

🧑 Авторы корпоративных программ по AI-driven разработке для ИТ-команд

🏆 Победители и призеры международных AI-хакатонов

🧑 Призеры LLM-coding challenge 2025

<> 20+ лет в разработке ПО, 10+ лет в области ИИ

⚡ Сооснователи лаборатории AIRnD.ru и авторы канала @AI.Dialogs

🧑 Спикеры на митапах и AI-мероприятиях, создатели @devclubspb

" Мы не только учим Cursor AI — мы практикуем каждый день

▶ YouTube канал → 💬 Telegram @aidialogs →



Сергей Смирнов

AI-эксперт и методолог, к.т.н.

Мастер системных и научно-практических подходов к построению AI-разработки, руководитель RnD лаборатории

23 года в Software Engineering, 15 лет в Computer Science, 4 года в GenAI

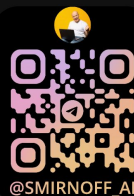


Александр Кожин

Архитектор GenAI решений

Практикующий эксперт по AI-driven разработке, эксперт в архитектуре сложных ИТ-систем

20+ лет разработки ПО от программиста до технического лидера



LLMSTART.RU

AIRND.RU

AGENTS

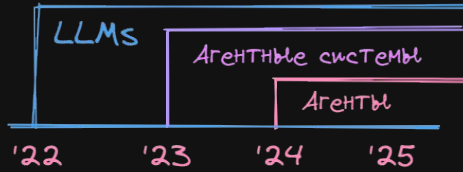
Agents

- основные понятия: memory, tools, reasoning
- паттерны

ИИ-агенты / Agentic AI



ИИ-агенты / Agentic AI



Монолитная LLM

переход

Агентные системы

⊖ знания огранич.
⊖ сложно адант.

Агентные системы

⊕ модульность
⊕ гибкость

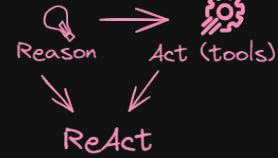
Программируемая (человек)

БЫСТРО

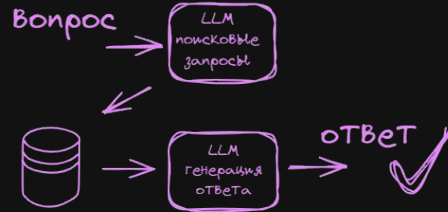
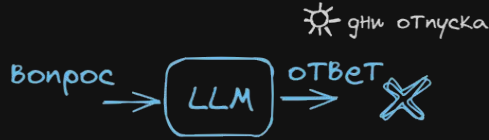
Агент (LLM)

МЕДЛЕННО

автоматность

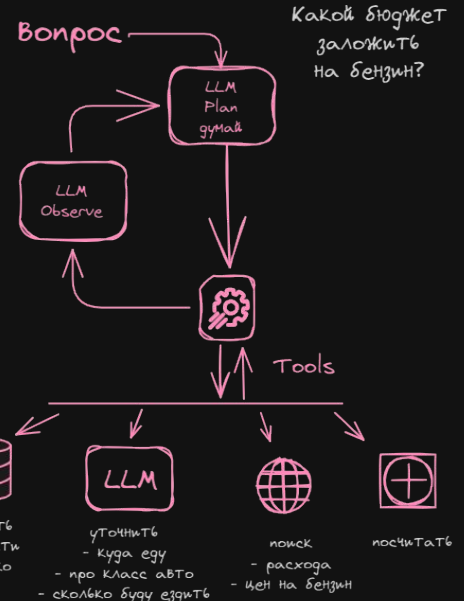


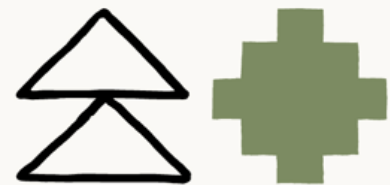
поиск
вычисления
когда погода
LLM
...



Retrieval-Augmented Generation

Pre-defined workflows





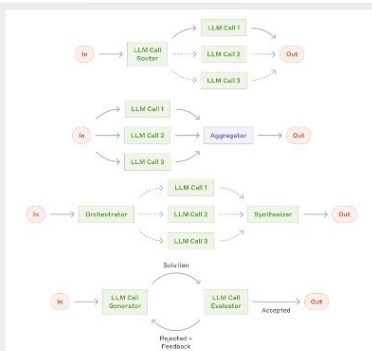
Building effective agents

<https://www.anthropic.com/engineering/building-effective-agents>



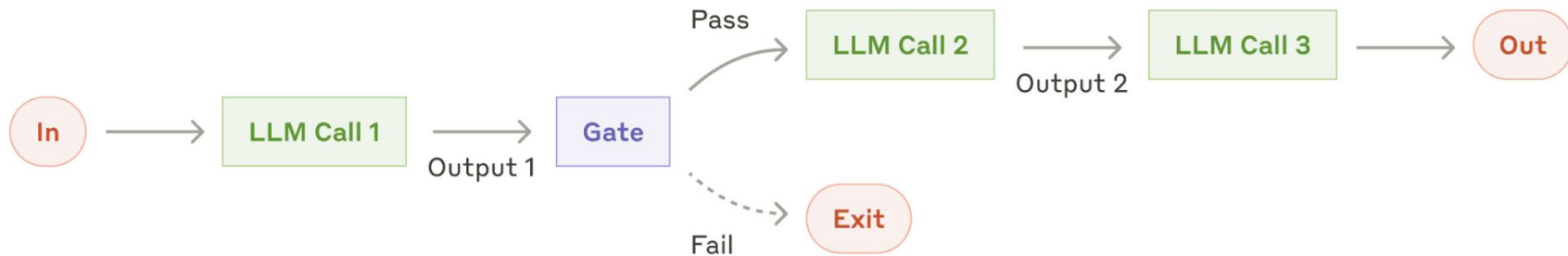
ИИ-агенты 2025 полный обзор: от основ до лучших практик и сценариев...

- Workflows are systems where LLMs and tools are orchestrated through predefined code paths.
- Agents, on the other hand, are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks.



https://www.youtube.com/watch?v=sYk_DygMFeA

Workflow: Prompt chaining



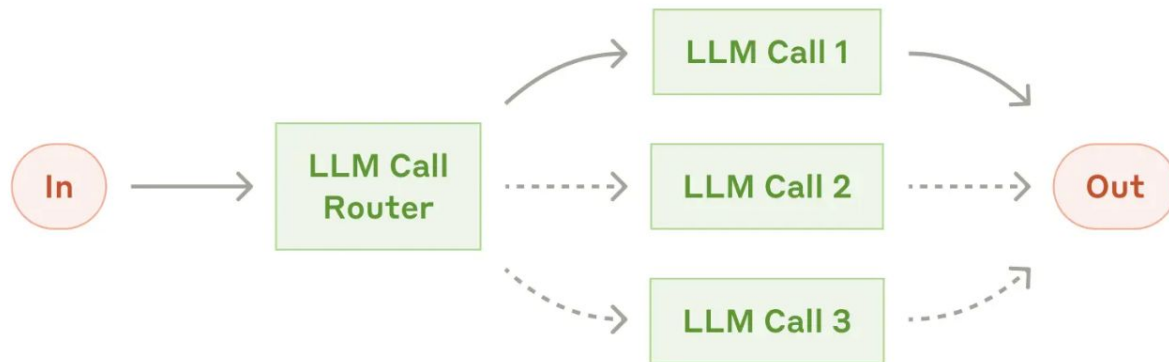
When to use?

- The task can be easily and cleanly decomposed into fixed subtasks.
- The main goal is to trade off latency for higher accuracy, by making each LLM call an easier task.

Examples

- Generating Marketing copy, then translating it into a different language.
- Writing an outline of a document, checking that the outline meets certain criteria, then writing the document based on the outline.

Workflow: Routing



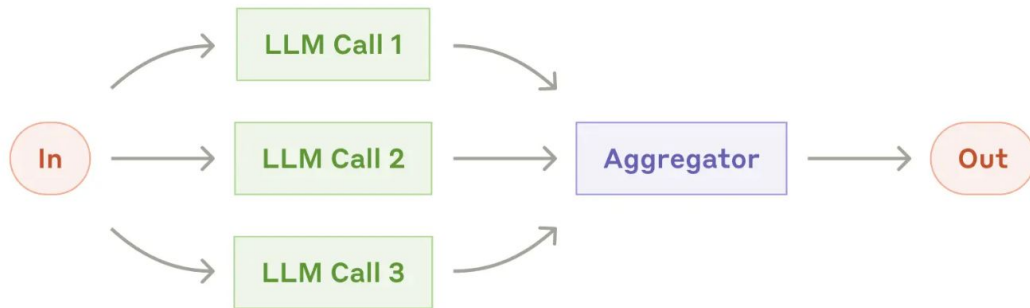
When to use?

- Complex tasks where there are distinct categories that are better handled separately,
- Where classification can be handled accurately, either by an LLM or a more traditional classification model/algorithm.

Examples

- Directing different types of customer service queries (general questions, refund requests, technical support) into different downstream processes, prompts, and tools.
- Routing easy/common questions to smaller models and hard/unusual questions to more capable models.

Workflow: Parallelization



When to use?

- Subtasks can be parallelized for speed, or when multiple perspectives or attempts are needed for higher confidence results.

Examples

- **Sectioning:** Breaking a task into independent subtasks run in parallel.
 - Each LLM call evaluates a different aspect
- **Voting:** Running the same task multiple times to get diverse outputs.
 - Several different prompts review and flag the code

Workflow: Orchestrator-workers



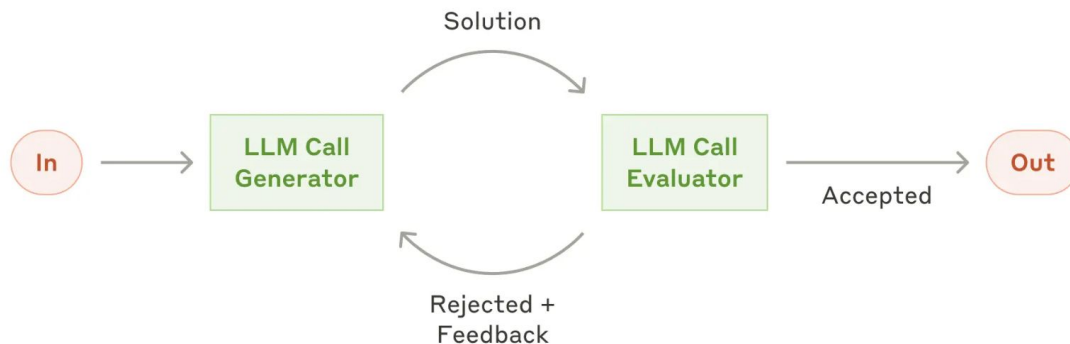
When to use?

- Complex tasks where you can't predict the subtasks needed.
- The **key difference from parallelization** is its flexibility: **subtasks aren't pre-defined**, but determined by the orchestrator based on the specific input.

Examples

- Coding products that make complex changes to multiple files each time.
 - Number of files differs and the nature of the changes depend on the task
- Search tasks that involve gathering and analyzing information from multiple sources for possible relevant information.

Workflow: Evaluator-optimizer



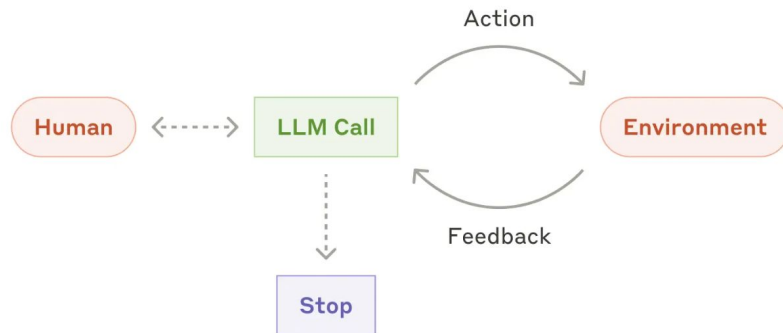
When to use?

- Clear evaluation criteria
- Iterative refinement provides measurable value.
- LLM responses can be demonstrably improved with human feedback
- LLM can provide such feedback.
- Like iterative writing process a

Examples

- Literary translation.
- Complex search tasks that require multiple rounds of searching and analysis

Autonomous agent



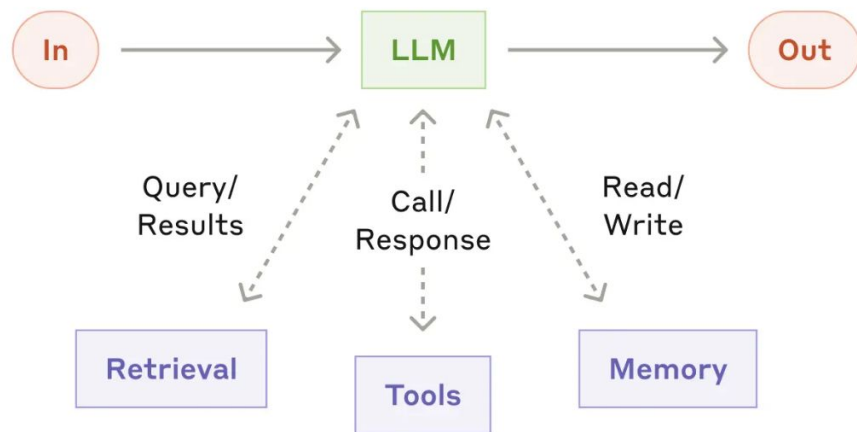
When to use?

- Open-ended problems - it's difficult or impossible to predict and hardcode.
- Must have some level of trust in its decision-making
- It means higher costs, and the potential for compounding errors

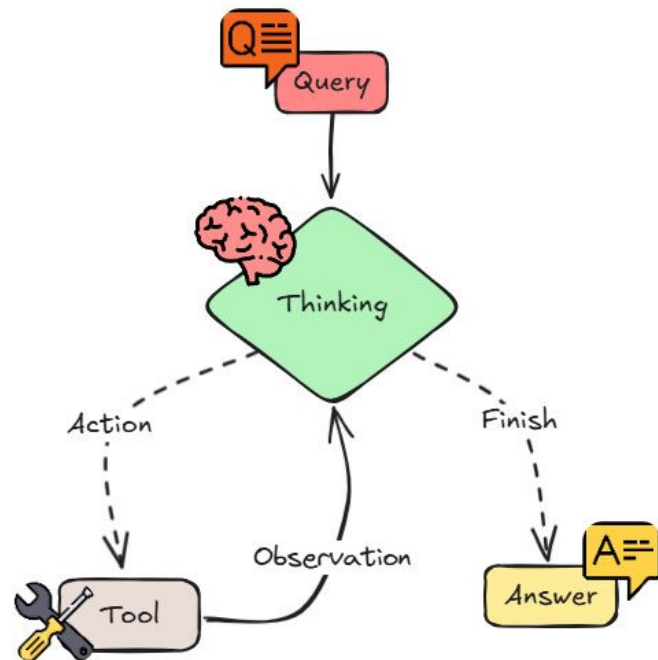
Examples

- A coding Agent to resolve SWE-bench tasks,
- Anthropic "computer use" reference implementation, where Claude uses a computer to accomplish tasks.
- Customer support/selling chat-bot

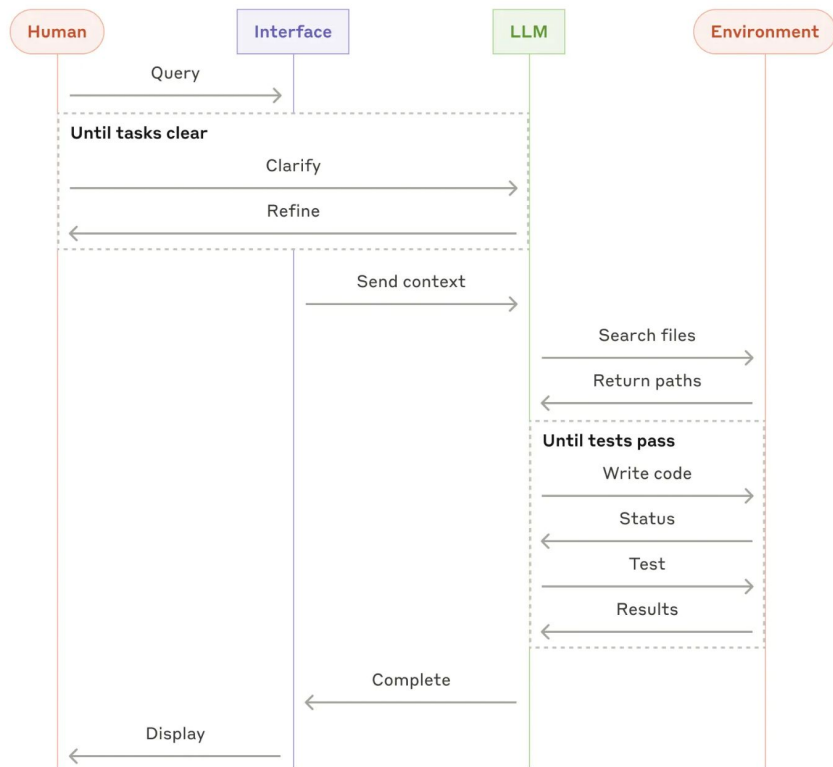
Summary: agent basic



ReAct Agent architecture



High-level flow of a coding agent



Summary from Anthropic

AI

Summary

Success in the LLM space isn't about building the most sophisticated system. It's about building the *right* system for your needs. Start with simple prompts, optimize them with comprehensive evaluation, and add multi-step agentic systems only when simpler solutions fall short.

When implementing agents, we try to follow three core principles:

1. Maintain **simplicity** in your agent's design.
2. Prioritize **transparency** by explicitly showing the agent's planning steps.
3. Carefully craft your agent-computer interface (ACI) through thorough tool **documentation and testing**.

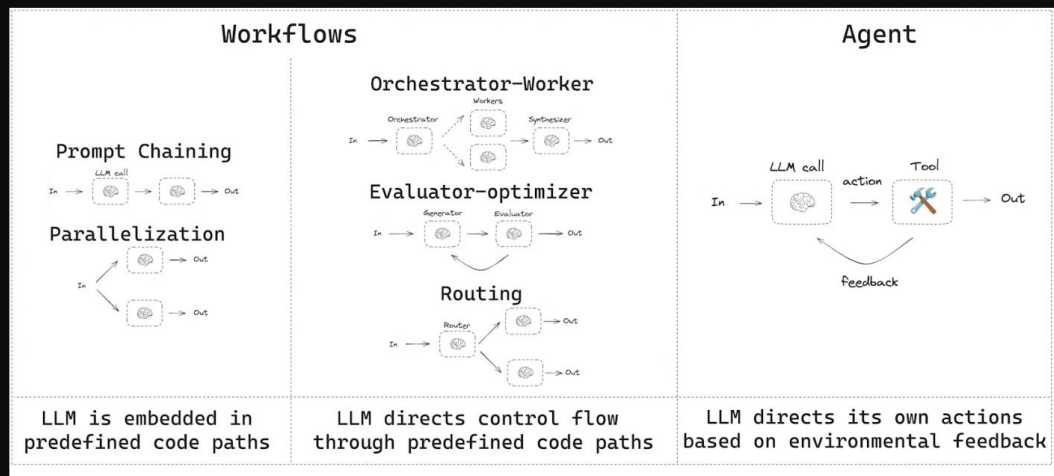
Frameworks can help you get started quickly, but don't hesitate to reduce abstraction layers and build with basic components as you move to production. By following these principles, you can create agents that are not only powerful but also reliable, maintainable, and trusted by their users.

Workflows and agents

Copy page

This guide reviews common workflow and agent patterns.

- Workflows have predetermined code paths and are designed to operate in a certain order.
- Agents are dynamic and define their own processes and tool usage.



Agents

- технические аспекты

Structured Output

Техники

- Constrained Decoding
- Format-Restricting Instructions
- NL-to-Format Conversion

Реализации

- Облачные
- Локальные: LangChain, BAML, Instructor, TypeChat

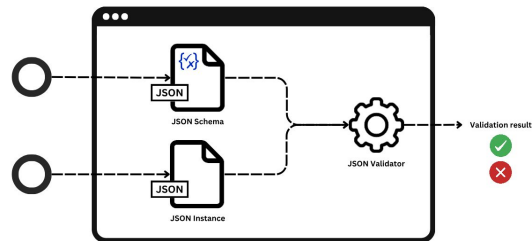
Ограничения

- Галлюцинации
- Компромисс рассуждения
- Ограниченная поддержка

Getting a structured response

python 

```
1 from openai import OpenAI
2 from pydantic import BaseModel
3
4 client = OpenAI()
5
6 class CalendarEvent(BaseModel):
7     name: str
8     date: str
9     participants: list[str]
10
11 response = client.responses.parse(
12     model="gpt-4o-2024-08-06",
13     input=[
14         {"role": "system", "content": "Extract the event information."},
15         {
16             "role": "user",
17             "content": "Alice and Bob are going to a science fair on Friday.",
18         },
19     ],
20     text_format=CalendarEvent,
21 )
22
23 event = response.output_parsed
```





Вызов функций (Function calling / Tool calling)

LLM умеет определять, когда необходимо вызвать функцию и создаёт параметры в виде JSON — на основе естественного языка.

 **Принцип работы** - модель не исполняет код, а генерирует JSON с:

-  именем функции
-  параметрами



Назначение

-  Извлечение структурированных данных
-  Выполнение действий



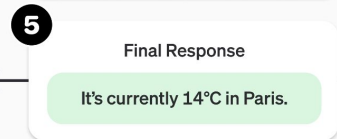
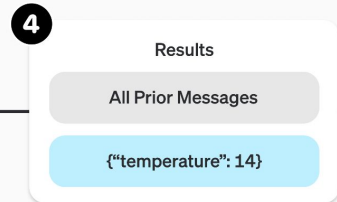
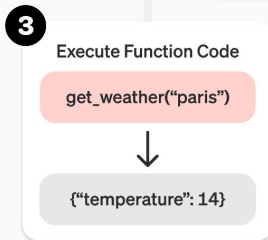
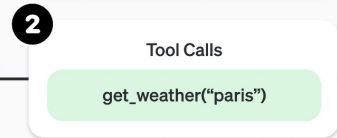
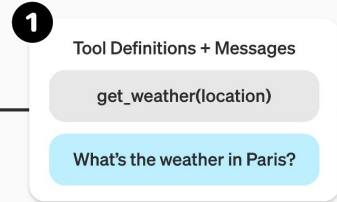
Вызовы

-  Разные реализации у провайдеров
-  Неустойчивость в сложных сценариях
-  Разрыв между open-source и коммерческими LLM

```
1 from openai import OpenAI
2
3 client = OpenAI()
4
5 tools = [{
6     "type": "function",
7     "name": "get_weather",
8     "description": "Get current temperature for a given location.",
9     "parameters": {
10         "type": "object",
11         "properties": {
12             "location": {
13                 "type": "string",
14                 "description": "City and country e.g. Bogotá, Colombia"
15             }
16         },
17         "required": [
18             "location"
19         ],
20         "additionalProperties": False
21     }
22 }]
23
24 response = client.responses.create(
25     model="gpt-4.1",
26     input=[{"role": "user", "content": "What is the weather like in Paris today?"}],
27     tools=tools
28 )
29
30 print(response.output)
```

Developer

Model



ⓘ While these LangChain classes support the indicated advanced feature, you may need to refer to provider-specific documentation to learn which hosted models or backends support the feature.

Model	Tool calling	Structured output	JSON mode	Local	Multimodal
ChatAnthropic	✓	✓	✗	✗	✓
ChatOpenAI	✓	✓	✓	✗	✓
AzureChatOpenAI	✓	✓	✓	✗	✓
ChatVertexAI	✓	✓	✗	✗	✓
ChatGoogleGenerativeAI	✓	✓	✗	✗	✓
ChatGroq	✓	✓	✓	✗	✗
ChatBedrock	✓	✓	✗	✗	✗
ChatHuggingFace	✓	✓	✗	✓	✗
ChatOllama	✓	✓	✓	✓	✗
ChatWatsonx	✓	✓	✓	✗	✓
ChatXAI	✓	✓	✗	✗	✗
ChatNVIDIA	✓	✓	✓	✓	✓
ChatCohere	✓	✓	✗	✗	✗
ChatMistralAI	✓	✓	✗	✗	✗
ChatTogether	✓	✓	✓	✗	✗
ChatFireworks	✓	✓	✓	✗	✗
ChatLlamaCpp	✓	✓	✗	✓	✗
ChatDatabricks	✓	✓	✗	✗	✗
ChatPerplexity	✗	✓	✓	✗	✓

Anthropic: Prompt engineering tools tips



One rule of thumb is to think about how much effort goes into human-computer interfaces (HCI), and plan to invest just as much effort in creating good *agent*-computer interfaces (ACI). Here are some thoughts on how to do so:

- Put yourself in the model's shoes. Is it obvious how to use this tool, based on the description and parameters, or would you need to think carefully about it? If so, then it's probably also true for the model. A good tool definition often includes example usage, edge cases, input format requirements, and clear boundaries from other tools.
- How can you change parameter names or descriptions to make things more obvious? Think of this as writing a great docstring for a junior developer on your team. This is especially important when using many similar tools.
- Test how the model uses your tools: Run many example inputs in our [workbench](#) to see what mistakes the model makes, and iterate.
- Poka-yoke your tools. Change the arguments so that it is harder to make mistakes.

Agents

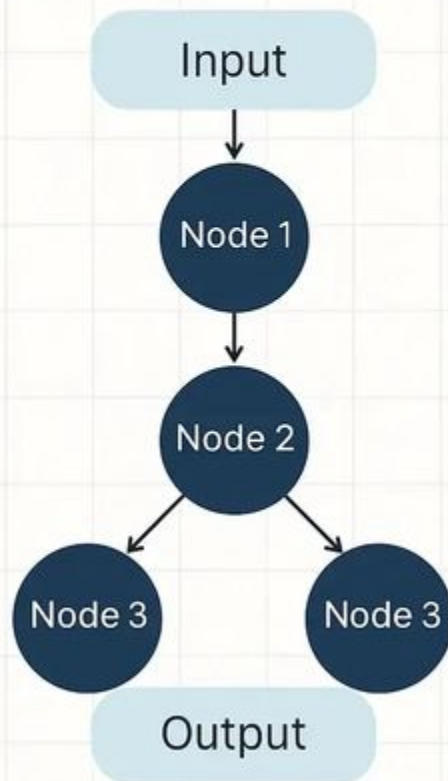
- фреймворки

LANGCHAIN VS LANGGRAPH

Execution Flow

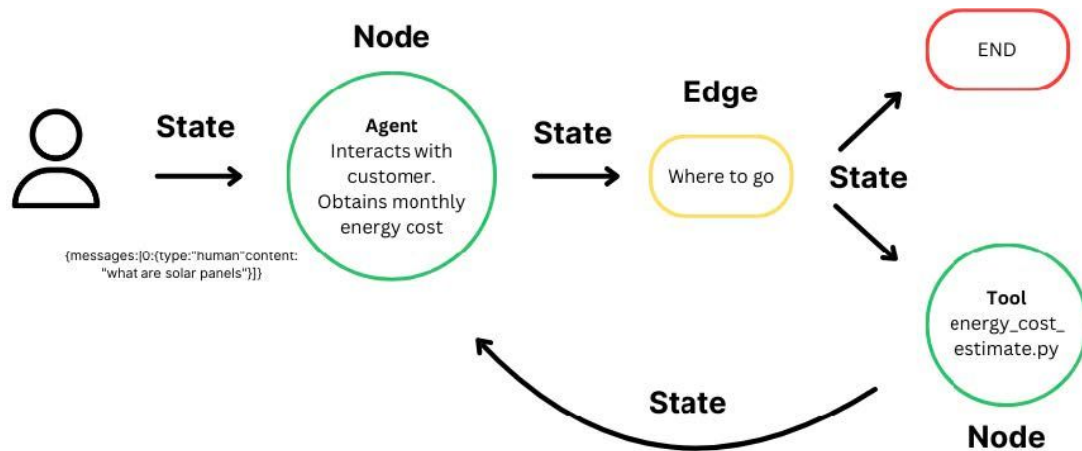


Execution Flow



LangGraph

★ 12k

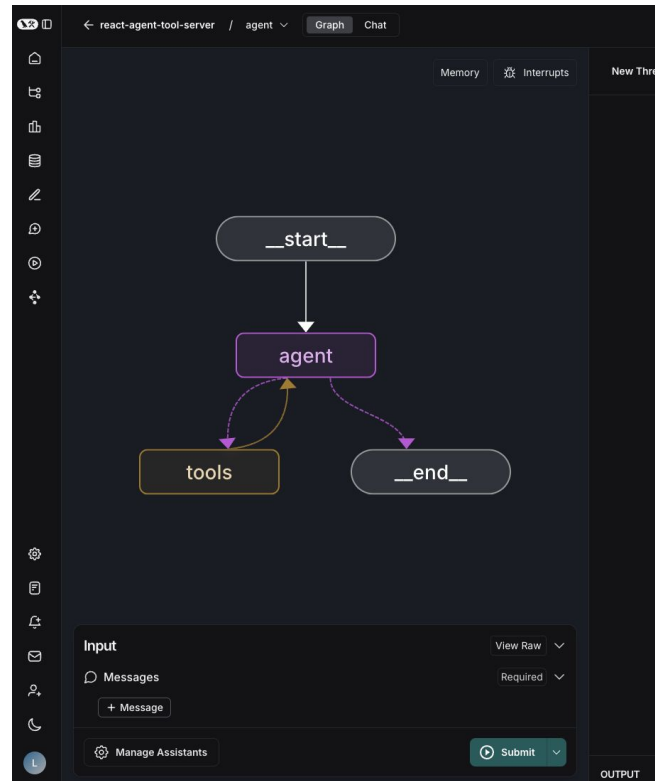


Thinking in LangGraph

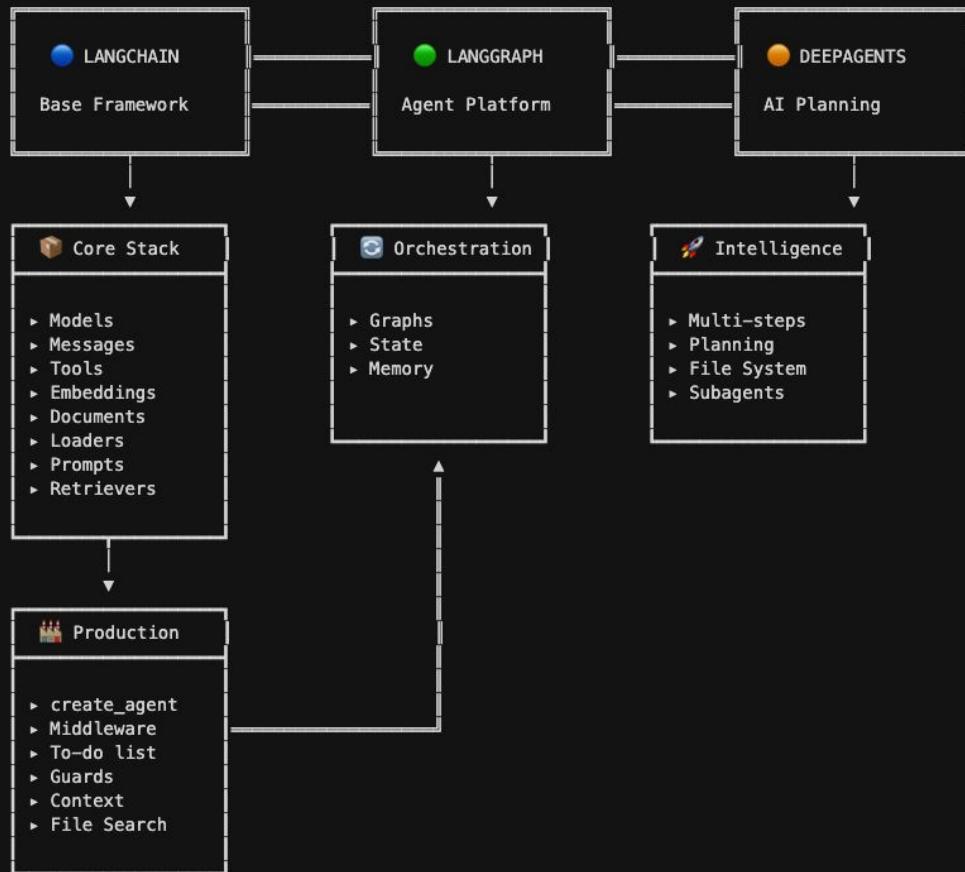
Copy page

Learn how to think about building agents with LangGraph by breaking down a customer support email agent into discrete steps

<https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph>



LangChain Ecosystem Overview



Get started

Philosophy

 Copy page



LangChain exists to be the easiest place to start building with LLMs, while also being flexible and production-ready.

LangChain is driven by a few core beliefs:

- Large Language Models (LLMs) are great, powerful new technology.
- LLMs are even better when you combine them with external sources of data.
- LLMs will transform what the applications of the future look like. Specifically, the applications of the future will look more and more agentic.
- It is still very early on in that transformation.
- While it's easy to build a prototype of those agentic applications, it's still really hard to build agents that are reliable enough to put into production.

Всем AI-успехов

Смирнов Сергей

Кожин Александр

<https://airnd.ru>, <https://llmstart.ru>

консалтинг и разработка
ИИ-решений для бизнеса

